Laboratorio 3: Transición de Paradigmas De la Verbosidad Estructural a la Concisión Moderna

1. Introducción

Este laboratorio se enfoca en la transición hacia la **concisión moderna** y la **seguridad estructural** de Kotlin.

2. Objetivos de Aprendizaje

- **Evaluar** el impacto de la verbosidad de Java en la mantenibilidad del código.
- **Implementar** modelos de datos inmutables y seguros contra nulos en Kotlin.
- **Sintetizar** lógica condicional compleja utilizando when como expresión.
- **Configurar y ejecutar** flujos de compilación de Kotlin mediante la interfaz de línea de comandos (CLI).

3. Contexto: Sistema de Gestión de Inventarios (SGI)

El sistema para desarrollar debe gestionar productos para una tienda tecnológica. Cada producto tiene un identificador, un nombre y un precio. El núcleo del sistema es la **clasificación automática**:

- **Categoría A (Premium):** Precio > $500.
- **Categoría B (Estándar):** Precio entre $100 y $500.
- **Categoría C (Económico):** Precio < $100.

El objetivo es comparar cómo ambos lenguajes manejan la creación de objetos, la entrada de datos del usuario y la seguridad ante valores nulos.

4. Requisitos Previos

- JDK instalado y configurado.
- Compilador de Kotlin (kotlinc) instalado.
- Editor de texto básico (VS Code o Notepad++).

5. Paso a Paso Detallado

Los estudiantes desarrollarán un programa funcional en Java que evidencie la carga de trabajo manual y los riesgos de nulidad. Luego, realizarán una refactorización total en Kotlin, utilizando características modernas para garantizar la robustez del software.

Parte A: El Legado de Java (Verbosidad y Riesgo)

En esta fase, el estudiante experimentará la "carpintería sintáctica" necesaria para que un objeto funcione correctamente en Java.

Paso 1: Definición del Modelo (Producto.java) Debe escribir aproximadamente 50 líneas de código para un modelo simple.

- **Subpaso 1.1 (Atributos):** Declarar id, nombre y precio.
- **Subpaso 1.2 (Constructor):** Crear el constructor para inicializar los campos.
- **Subpaso 1.3 (Boilerplate):** Escribir manualmente los *Getters* y *Setters*.
- **Subpaso 1.4 (Infraestructura):** Sobrescribir toString(), equals() y hashCode(). Sin esto, las colecciones y el depurador no funcionarán correctamente.

Paso 2: Lógica de Interacción (GestionJava.java)

- **Subpaso 2.1:** Usar la clase Scanner para solicitar los datos al usuario.
- **Subpaso 2.2 (Riesgo NPE):** Intentar convertir el nombre ingresado a mayúsculas. ¿Qué pasa si el usuario presiona Enter sin escribir nada y el sistema lo recibe como null? Java lanzará un NullPointerException en tiempo de ejecución.
- **Subpaso 2.3 (Control de Flujo):** Implementar un switch para la clasificación. Recordar que en Java el switch es una **sentencia** (statement) y requiere break para evitar el error de "caída en cascada" (fall-through).
- **Subpaso 2.4 (Prueba de Infraestructura):** En el método `main`, crear dos objetos `Producto` con los **mismos datos**.
    - Imprimir ambos productos directamente: `System.out.println(p1);`. (Esto obliga a usar `toString`).
    - Comparar con `==` e imprime el resultado.
    - Comparar con `.equals()` e imprime el resultado.

- Crear un `java.util.HashSet<Producto>` y añade el primer objeto. Intentar buscar el segundo objeto usando `set.contains(p2)`. (Esto obliga a usar `hashCode`).

Parte B: La Evolución con Kotlin (Concisión y Seguridad)

Aquí aplicará la filosofía *Kotlin-first* para reducir el ruido sintáctico y garantizar la seguridad estática. Todo el código descrito a continuación se debe colocar en un mismo archivo llamado Inventario.kt. Para construir su archivo Inventario.kt, ensamblar los siguientes bloques de código en orden. Evitar el copiar y pegar masivo; analizar qué hace cada línea.

Paso 1: Construcción del archivo (Inventario.kt)

- **Subpaso 1.1: Importación y Modelo de Datos** Comenzamos importando la herramienta de lectura de Java (interoperabilidad) y definiendo nuestra entidad. Una `data class` en Kotlin equivale a las 50 líneas de boilerplate que escribió en Java.

```
import java.util.Scanner

// La data class genera automáticamente: equals(), hashCode(), toString() y
copy()
data class Producto(val id: String, val nombre: String, val precio: Double)
```

- **Subpaso 1.2: Punto de entrada y preparación del Lector** Todo programa ejecutable necesita una función main. Aquí inicializamos el Scanner utilizando la sintaxis de Kotlin (notarás que System.in`` lleva comillas invertidas porque in es una palabra reservada en Kotlin).

```
fun main() {
    val lector = Scanner(System.`in`)
    println("--- Registro de Inventario Kotlin ---")
```

- **Subpaso 1.3: Lectura de datos y manejo de Nulidad** Aquí aplicamos Inferencia de Tipos (no declaramos String o Double). Para el nombre, usamos readLine() que puede devolver nulo; el operador Elvis (?:) asegura que, si la entrada es nula, el programa asigne un valor por defecto en lugar de fallar.

```
    print("Ingrese ID del producto: ")
    val idInput = lector.next()

    print("Ingrese Nombre del producto: ")
    // readLine() devuelve String?. El operador ?: evita el NPE
    val nombreInput = readLine() ?: "Sin Nombre"

    print("Ingrese Precio: ")
    val precioInput = lector.nextDouble()
```

- **Subpaso 1.4: Instanciación y Clasificación con when** En Kotlin no usamos la palabra new. Además, usamos when como una expresión, lo que nos permite asignar el resultado de la evaluación directamente a una variable inmutable.

```
    val p1 = Producto(idInput, nombreInput, precioInput)

    // 'when' como expresión: evalúa y retorna un valor inmediatamente
    val categoria = when {
        p1.precio > 500.0 -> "Premium"
        p1.precio in 100.0..500.0 -> "Estándar"
        else -> "Económico"
    }
```

- **Subpaso 1.5: Salida de datos y Comparación Estructural** Finalmente, imprimimos los resultados. Verá que al imprimir p1, Kotlin usa el toString() generado automáticamente. Al final, comparamos dos objetos con == (que en Kotlin llama a .equals() de forma segura).

```
    println("\nProducto Registrado con éxito:")
    println(p1) //Formato legible automático: Producto(id=..., nombre=...,
precio=...)
    println("Categoría asignada: $categoria")

    // Prueba de igualdad de contenido (infraestructura automática)
    val p2 = Producto(idInput, nombreInput, precioInput)
    println("\n¿Contenido idéntico?: ${p1 == p2}")
}
```

Paso 3: Guía de Compilación en Windows (CLI) Los estudiantes aprenderán que Kotlin no se ejecuta directamente, sino que se traduce a bytecode para la JVM.

1. **Descargar e Instalar** Kotlinc en su PC (Install the compiler)
2. **Abrir CMD** en la carpeta donde guardó Inventario.kt.
3. **Compilar:** Ejecutar el siguiente comando para generar el ejecutable .jar:
   `kotlinc Inventario.kt -include-runtime -d Inventario.jar`
   - **-include-runtime**: Incluye las librerías necesarias de Kotlin en el archivo resultante.
   - **-d Inventario.jar**: Indica el nombre del archivo de destino.

3. **Ejecutar:** Correr el programa sobre la máquina virtual de Java:

   `java -jar Inventario.jar`

6. Resultados Esperados

- **Código Limpio:** El archivo Kotlin debe ser significativamente más legible al eliminar el boilerplate. El archivo Java tendrá cerca de 80 líneas; el de Kotlin tendrá menos de 25.
- **Seguridad:** El programa Kotlin no debe "crashear" ante entradas nulas si se usaron correctamente los operadores de seguridad. El uso de readLine() ?: "Sin Nombre" previene cierres inesperados por nulidad.
- **Inmutabilidad:** Uso predominante de val sobre var.
- **Consistencia**: El HashSet de Java solo funcionará si los estudiantes implementaron correctamente hashCode. En Kotlin, esto "simplemente funciona" gracias a la data class.

7. Desafío Extra: El Botón de Pánico

Problema: Intentar forzar un error en Kotlin usando el operador de aserción no nula **!!** en la entrada de datos. Reemplazar la línea del nombre por: `val nombreInput = readLine()!!` Ejecutar el programa y, cuando pida el nombre, presionar Ctrl + C o intentar simular una entrada nula.

```
val nombreForzado = readLine()!! // Si la entrada es nula, lanzará un NPE inmediato.
```

***Reflexión**:*

- ¿Por qué este operador se considera un "Code Smell" en producción?
- ¿Es responsable usar !! en una aplicación que será usada por millones de personas?

8. Preguntas de Reflexión

Responda a las siguientes preguntas basándose en su experiencia durante el laboratorio y el material teórico:

- **Inmutabilidad**: ¿Cómo beneficia el uso de val en lugar de var a la seguridad de una aplicación móvil que maneja múltiples hilos de ejecución (concurrencia)?
- **Costo del Error**: Explique el concepto de "Fallar Rápido" (Fail-Fast) y cómo el sistema de tipos de Kotlin traslada el costo del error del tiempo de ejecución al tiempo de compilación.
- **Abstracción**: En la Parte A (Java), ¿qué sucede con la búsqueda de un objeto en un HashSet si se olvida sobrescribir el método hashCode? ¿Cómo resuelve Kotlin esto automáticamente?

- **Control de Flujo**: ¿Cuál es la ventaja técnica de que when sea una expresión que devuelve un valor, en comparación con el switch de Java que es una sentencia?
- **Interoperabilidad**: Si usted tuviera que migrar una aplicación grande de Java a Kotlin de forma gradual, ¿qué precauciones debería tomar al manejar los llamados Platform Types (ej. String!)?

9. Entregables

- **Informe:** con capturas de pantallas pertinentes, respuestas a preguntas, conclusiones y recomendaciones, bibliografía, etc.
- **Código fuente:** archivos Producto.java, GestionJava.java e Inventario.kt.
- **Captura de pantalla** de la terminal mostrando la ejecución de ambos programas.
- **Explicación** del por qué Kotlin reduce la probabilidad de errores en comparación con la implementación manual de Java.

10. Rúbrica de Evaluación

- No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe.